

Лабораторная работа №5

Аппарат сетей Петри: задача обедающих философов

Назармамадов Умед Джамshedович

Содержание I

Цель работы

- Изучить аппарат сетей Петри

Цель работы

- Изучить аппарат сетей Петри
- Реализовать задачу «Обедающие философы»

Цель работы

- Изучить аппарат сетей Петри
- Реализовать задачу «Обедающие философы»
- Исследовать проблему взаимной блокировки (deadlock)

Цель работы

- Изучить аппарат сетей Петри
- Реализовать задачу «Обедающие философы»
- Исследовать проблему взаимной блокировки (deadlock)
- Изучить способ предотвращения тупика через арбитра

- Позиции (места) — хранят фишки, описывают состояния

Переход разрешён, если во входных позициях достаточно фишек.

- Позиции (места) — хранят фишки, описывают состояния
- Переходы — активные элементы, описывают события

Переход разрешён, если во входных позициях достаточно фишек.

- Позиции (места) — хранят фишки, описывают состояния
- Переходы — активные элементы, описывают события
- Дуги — связи между позициями и переходами

Переход разрешён, если во входных позициях достаточно фишек.

- Позиции (места) — хранят фишки, описывают состояния
- Переходы — активные элементы, описывают события
- Дуги — связи между позициями и переходами
- Маркировка — текущее распределение фишек

Переход разрешён, если во входных позициях достаточно фишек.

Задача обедающих философов

- N философов за круглым столом

Задача обедающих философов

- N философов за круглым столом
- Между соседями лежит одна вилка

Задача обедающих философов

- N философов за круглым столом
- Между соседями лежит одна вилка
- Для еды нужны обе соседние вилки

Задача обедающих философов

- N философов за круглым столом
- Между соседями лежит одна вилка
- Для еды нужны обе соседние вилки
- Сформулирована Дейкстрой в 1965 году

Проблема deadlock

При наивной реализации:

- Каждый философ сначала берёт левую вилку

Проблема deadlock

При наивной реализации:

- Каждый философ сначала берёт левую вилку
- Если все возьмут левую одновременно

Проблема deadlock

При наивной реализации:

- Каждый философ сначала берёт левую вилку
- Если все возьмут левую одновременно
- Все будут бесконечно ждать правую

При наивной реализации:

- Каждый философ сначала берёт левую вилку
- Если все возьмут левую одновременно
- Все будут бесконечно ждать правую
- Система замирает — взаимная блокировка

- Дополнительная позиция с $N-1$ фишками

Решение через арбитра

- Дополнительная позиция с $N-1$ фишками
- Ограничивает число философов, одновременно берущих вилки

Решение через арбитра

- Дополнительная позиция с $N-1$ фишками
- Ограничивает число философов, одновременно берущих вилки
- Теоретически никогда не заходит в deadlock

- Структура PetriNet с матрицей инцидентности

- Структура PetriNet с матрицей инцидентности
- `build_classical_network` — сеть без синхронизации

- Структура PetriNet с матрицей инцидентности
- `build_classical_network` — сеть без синхронизации
- `build_arbiter_network` — сеть с арбитром

- Структура PetriNet с матрицей инцидентности
- `build_classical_network` — сеть без синхронизации
- `build_arbiter_network` — сеть с арбитром
- Стохастическое моделирование методом Гиллеспи

Базовые эксперименты

```
umedn@HUAWEI:~/work/study/2026-1/2026-1-study-simulation-modeling/labs/Lab05$ julia --project=project
project/scripts/dining_philosophers.jl
=== Классическая сеть (без арбитра) ===
Deadlock обнаружен: true

=== Сеть с арбитром ===
Deadlock обнаружен: false
Готово!
umedn@HUAWEI:~/work/study/2026-1/2026-1-study-simulation-modeling/labs/Lab05$ |
```

$N=5$ философов, $t_{\max}=50$. Классическая сеть vs сеть с арбитром.

- Все философы переходят в состояние «Голоден»

Результаты: классическая сеть

- Все философы переходят в состояние «Голоден»
- Ни один не может поесть

Результаты: классическая сеть

- Все философы переходят в состояние «Голоден»
- Ни один не может поесть
- `detect_deadlock` возвращает `true`

- Постоянное чередование приёма пищи

Результаты: сеть с арбитром

- Постоянное чередование приёма пищи
- Отсутствие длительных блокировок

Результаты: сеть с арбитром

- Постоянное чередование приёма пищи
- Отсутствие длительных блокировок
- `detect_deadlock` возвращает `false`

Анимация процесса

```
umedn@HUAWEI:~/work/study/2026-1/2026-1-study-simulation-modeling/labs/lab05$ julia --project=project
project/scripts/dining_philosophers_animation.jl
[ Info: Saved animation to /home/umedn/work/study/2026-1/2026-1-study-simulation-modeling/labs/lab05/
project/plots/philosophers_simulation.gif
Анимация сохранена в plots/philosophers_simulation.gif
umedn@HUAWEI:~/work/study/2026-1/2026-1-study-simulation-modeling/labs/lab05$ |
```

Наглядная демонстрация изменения маркировки во времени.

Итоговый сравнительный график

- Классическая сеть: линии «Ест» падают до нуля и остаются там

Итоговый сравнительный график

- Классическая сеть: линии «Ест» падают до нуля и остаются там
- Сеть с арбитром: линии продолжают колебаться

Итоговый сравнительный график

- Классическая сеть: линии «Ест» падают до нуля и остаются там
- Сеть с арбитром: линии продолжают колебаться
- Количественное подтверждение эффективности арбитра

- Все скрипты преобразованы через Literate.jl

- Все скрипты преобразованы через Literate.jl
- Сгенерированы: чистый код, Quarto-документация, Jupyter Notebook

- Все скрипты преобразованы через Literate.jl
- Сгенерированы: чистый код, Quarto-документация, Jupyter Notebook
- Notebooks выполнены и проверены

- Реализована классическая задача синхронизации

- Реализована классическая задача синхронизации
- Подтверждена неизбежность deadlock без синхронизации

- Реализована классическая задача синхронизации
- Подтверждена неизбежность deadlock без синхронизации
- Арбитр полностью предотвращает взаимную блокировку

- Реализована классическая задача синхронизации
- Подтверждена неизбежность deadlock без синхронизации
- Арбитр полностью предотвращает взаимную блокировку
- Результаты наглядно продемонстрированы графиками и анимацией